
Proof-Carrying Neurosymbolic Supervision: State, Logic-Governed Decisions, and Test-Evidence Reuse

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Coding agents repeatedly reconstruct repository state, regenerate plans, and re-
2 run validation, yet shorter prompts or cached passes can omit dependencies and
3 weaken assurance. We present an obligation-centered architecture that maintains
4 content-addressed semantic state separately from operational task state and durable
5 evidence. A formalization layer maps code, intent, and applicable constraints to
6 typed obligations while exposing unsupported or lossy translations. A supervisor
7 uses these obligations to select deterministic analysis, compositional reasoning,
8 proof search, bounded synthesis, or an explicitly authorized neural residual. Candi-
9 date edits are checked against current source; test and proof results are admitted
10 only for their declared statements, environments, and trust profiles. Accepted
11 transitions update durable roots and invalidate affected planning and verification
12 state. The design includes incremental proof-unit sealing and staged fixture-aware
13 test reuse, semantic-world memory, and procedural synthesis. We specify separate
14 tests of preservation, progress, and end-to-end cost, including matched raw-context
15 baselines, cold validation oracles, and positive authorization cases. [RESULTS:
16 insert the frozen task population, useful completion rate, net measured cost, un-
17 certainty intervals, and false-admission results.] The contribution is a method for
18 governing repository evolution with explicit evidence, not a claim that hashing,
19 model confidence, or every available solver proves arbitrary Python correct.

20 1 Problem and contribution

21 The cost of an agent is not only the tokens used to generate a patch. It includes rediscovering relevant
22 code, reconstructing assumptions, choosing tests, recovering interrupted work, and deciding whether
23 previous evidence still applies. Repository agents and automated testing studies make the quality of
24 the agent–tool boundary central to useful repair [8, 11]. Here, that boundary is a persistent state-and-
25 obligation representation rather than an unstructured conversation history. Our proposed method has
26 three linked contributions. First, it compiles source and domain intent into identity-bound semantic
27 evidence and explicit unresolved obligations. Second, it makes planning, counterexample-guided
28 repair, and procedural synthesis consumers of those obligations, retaining neural models only for
29 unresolved proposals. Third, it separates test observations, proof statements, reuse decisions, and
30 accepted state transitions, enabling incremental work without silently upgrading evidence. Semantic
31 context reduction is one measurable consequence of this loop, not its sole purpose. The prototype
32 contains source-present components and incompletely qualified integration paths. This working paper
33 distinguishes those from methods specified in active workstreams and from results actually measured.
34 Source presence, branch landing, operational consumption, passing qualification,

35 Table 1: The semantic root, accepted-transition manifest, and mutable operational projection must
36 not be conflated.

37 State space	Authority and purpose	What it does not establish
----------------	-----------------------	----------------------------

38	Semantic state $G(F)$	Semantic compiler: program facts,	Task completion, a permission
39		contracts, dependency closure,	grant, or all-language behavioral
40		approximation, source references.	correctness.
41	Operational state O	Supervisor: goals, tasks, plan epochs,	A new meaning for a semantic
42		leases, attempts, outcomes, route and	object or proof correctness by
43		acceptance decisions.	bookkeeping.
44	Durable artifacts B	Store: canonical bytes, candidate	Truth, fresh applicability, or
45		indexes, proof forests, WAL and	authorization merely from
46		current-root CAS.	persistence.

47 and policy promotion are different statuses. The final artifact must freeze the exact multi-repository
48 source forest and the methods it exercises. [D1, A1–A5, N1–N14]

49 2 State and evidence are typed, separate authorities

50 2.1 The persistent state model

51 Let F be a frozen repository forest, including source trees, declared overlays, dependency pins, and
52 relevant environment identities. The incremental semantic index extracts stable symbols, semantic
53 versions, typed relationships, and limitations. A separate semantic-state producer seals the finalized
54 index into immutable facts, links, capsules, and environment bindings. It supplies pure selection and
55 freshness operations; it does not execute tests, operate a task queue, or decide that a candidate is
56 accepted. [D1]

57 2.2 Claims carry provenance and applicability

58 The domain-neutral IR kernel separates declarations, runs, results, and receipts. A declaration does
59 not acquire a different semantic identity because a model or solver ran. Canonical identities bind a
60 schema/domain and canonicalization profile; operational roots can reference declarations and their
61 histories without inserting attempts, clocks, or leases into semantic content. Legal, Security, and
62 Intent families share these protocols but do not share verdict authority or ontology. [N4]

$$e = (\text{claim, subjects, assumptions, scope, dependencies,} \quad (1) \\ \text{translation profile, checker result, environment, policy, provenance}).$$

63 Admission checks integrity, source binding, current applicability, required checker evidence, and
64 policy. These dimensions are not one scalar confidence score. A proof of an encoded property is
65 not permission to perform an action; a signed statement is not independently observed execution; a
66 candidate stored under a CID is not admitted knowledge. The hypothesis is that exact identities and
67 explicit conditions enable reliable reuse—not that cryptographic hashes establish universal semantic
68 equivalence.

69 2.3 Incrementality with a visible unknown frontier

70 Previous and current semantic states drive additive invalidation and test/proof selection. Fresh exact
71 dependency capsules may substitute for unchanged source under a declared profile; edited targets,
72 stale records, and opaque or insufficient analyses require exact source or broader validation. Dynamic
73 dispatch, reflection, fixtures, plugins, external services, and generated code can prevent a complete
74 dependency closure. Unknown edges widen the frontier rather than disappear from it. [D1; W1–W2]

75 Table 2: Formal tools are selected by the obligation and evidence profile; not every tool runs on every
76 task.

77 Formal mechanism	Question it answers	Required qualification
78 Abstract interpretation	Which values, effects, or paths 79 remain possible?	Sound over-approximation for the supported fragment; opacity 80 elsewhere.
81 Assume-guarantee	Do producer guarantees meet 82 consumer assumptions?	Typed implication, nonvacuity, and valid induction for cycles.
83 SMT / interpolation	Is an encoded obligation satisfied 84 or refuted; what boundary fact is 85 useful?	Exact theory, formulas, bounds, solver, result checks and translation profile.
86 Tactician / Hammer / 87 proof assistant	Which premises, tactics, or proof candidates close a hole?	Candidate search separated from reconstruction or kernel checking.

88	Temporal / deontic /	Which state, action, obligation,	A property-preserving encoding and
89	cognitive / F-logic views	or knowledge relation is being	an available compatible backend.
90		expressed?	
91	ZK relation / attestation	Which private witness satisfies	Circuit/program, public inputs, keys
92		which public statement?	and verifier; no inferred execution
93			claim.

94 **3 Formalization: preserve the property before proving it**

95 **3.1 Domain intent is not execution authority**

96 A high-level objective is first a candidate specification. Intent IR expresses goals, actions, resources,
97 and effects; Legal IR contributes applicable normative constraints; Security IR contributes principal,
98 resource, and effect invariants. Their shared IR kernel supplies canonical claims and evidence
99 references, not an automatic equivalence between domains. Reviewed formalization and source
100 mappings turn a candidate into bounded obligations; ambiguity, missing coverage, and unsupported
101 modalities remain explicit. [N4–N7] The attested authorization design queries a manifest-bound proof
102 corpus, checks applicability and revocation, composes obligations, and produces an exact-context
103 decision. Correctness and authoriza- tion are separate gates. The inspected reference enforcement
104 layer supports one-time consumption and context rechecks, but does not itself connect to real tools;
105 the documented receipt-consumption rollout defaults off/shadow. Thus the paper claims an interface
106 and mechanism only at the deployment stage actually exercised. [N7, N11]

107 **3.2 Loss-aware translation and solver-state limits**

108 A target formula is useful only for claims preserved by its translation. For example, a legacy
109 projection removes temporal/deontic operators and approximates a binary temporal formula by
110 conjunction. A proof of that projection cannot prove the original obligation. The admitted route
111 must reject unsupported loss or bind an independently justified preservation relation. Translation
112 validation checks the particular transformation rather than trusting every conversion [4]. [N8] The
113 inspected incremental SMT wrapper maintains named assertions and replay identity but rebuilds
114 a Z3 solver per check; that path supplies no exported proof object. It therefore supports assertion-
115 level incrementality, not an established retained-clause speedup. Solver diversity, same-solver core
116 rechecking, native proof reconstruction, and kernel checking must be reported separately. [D2]
117 The actual component checker lowers a producer guarantee G and consumer assumption A to $G \wedge$
118 $\neg A$; SAT supplies a countermodel, UNSAT discharges the encoded implication, and unsupported
119 clauses remain uncovered. The bounded QF_LIA CEGAR adapter searches a Boolean abstraction,
120 concretizes an error trace, and refines only spurious traces using validated predicates. A feasible trace
121 refutes the model; incomplete search is not a proof. These results become reasoned plan obligations
122 and localized repair inputs. [N12–N13]

123 **4 Planning and state repair before neural execution**

124 **4.1 Objective compilation and plan admission**

125 The direct-objective workstream replaces manual prompt elaboration with a bounded compiler:
126 authorized input and repository scan yield goals, subgoals, task dependencies, assumptions, permitted
127 effects, exact outputs, and acceptance obligations. Deterministic templates cover known task families.
128 A model may propose a residual decomposition, but the result must pass schema, dependency, scope,
129 resource, and acceptance-coverage checks. An inferred specification is still a proposal until admitted;
130 a model must not lower the objective or remove a required test to make its own plan succeed. [W2;
131 DOEP] Planning maintains an obligation frontier. Resolved obligations reference current evidence;
132 unre- solved obligations carry a reason and a permissible next route. Hard authority, safety, and
133 effect conditions are checked before ranking by cost. The intended routing policy selects the cheapest
134 adequate route, not a ritual invocation of every theorem prover.

135 current evidence / applicable procedure

136 $\rightarrow$ exact dependency and static analysis

137 $\rightarrow$ contract, abstract-state and solver checks

138 → bounded deterministic repair or synthesis
139 → explicitly authorized residual model
140 → independent validation / review / honest insufficiency

141 **4.2 A routing gate is not a proof kernel**

142 The pre-implementation kernel can defer unavailable capabilities, choose a unique analytical can-
143 didate, abstain on ambiguous candidates, or authorize a named neural residual using resolved task/
144 forest-bound receipts. It does not run a model, prove a candidate, or by itself complete a task. A
145 Boolean “closes claim” on a candidate requires independent downstream justification. Its local zero
146 provider-hook count is not a measurement of saved model calls across the workflow. [A1, N1]
147 Integration is part of the method. The real caller must bind the current plan, diagnostic view and
148 obligation graph to the same task and forest as its evidence. A valid authorized residual must progress
149 while a minimally altered invalid request fails. Measuring only rejection would reward a deny-all
150 agent rather than useful neurosymbolic control.

151 **4.3 Event-driven reassessment and bounded convergence**

152 Repository, dependency, policy, proof, fixture, capability, and owner events invalidate only the
153 affected causal/semantic frontier when that frontier is complete. The supervisor retains accepted
154 unaffected subplans and emits a PlanDelta rather than rebuilding an entire prompt. Refill creates
155 only tasks needed for explicit gaps or counterexamples, subject to plan epochs, generation limits,
156 repeated-candidate detection, and authority. Unknown provider effects enter reconciliation rather
157 than blind retry. [W2; DOEP] Queue exhaustion is not success. Completion requires current evidence
158 covering every mandatory acceptance obligation and no unresolved required consumer. Bounded
159 termination is also not convergence to a solution: budget exhaustion, nonconvergence, unavailable
160 tools, and human decisions remain separate terminal outcomes. Incoming world-model self-hosting
161 is qualified only when route and acceptance records show actual consumption of state, not merely
162 sidecar logging. [W1–W2]

163 **5 Repair, context compilation, and procedural synthesis**

164 **5.1 Localize and preserve the intended change**

165 A repair begins with a state-bound failed obligation, observation, or counterexample—not a free-form
166 request to “fix everything.” Static dependency analysis and abstract interpretation bound the affected
167 region. Assume–guarantee checks expose unmet component interfaces; validated unsat cores or
168 interpolants can identify a compact missing premise. CEGAR refines an abstraction after a spurious
169 counterexample; CEGIS refines candidate implementations against accumulated counterexamples [3,
170 5]. Conflating these loops would obscure whether the program or its abstraction changed. Candidate
171 plans, patch sketches, and CodeEditPackets remain non-authoritative. The currently represented path
172 checks preimages and scope, applies a Git patch in an isolated fenced worktree, rescans semantic
173 state, and runs selected plus required fallback validation. General tokenizer-alias IR editing and exact
174 CST/AST relinking remain separate PCSM extensions until their actual resolver, renderer, round-trip
175 and adversarial checks are frozen. A source map alone is not a proof of semantic preservation.

176 **5.2 Context minimizes unresolved information, not obligations**

177 The semantic compiler owns semantic ContextPack identity and sufficiency; the runtime compiles
178 provider-facing ContextCapsules. The goal, authority, scope, and acceptance core is non-truncatable.
179 Optional evidence is chosen under byte/token budgets; exclusions have exact expansion references
180 and reason codes. Required target source, caveats, tests, and proof obligations cannot be removed
181 to hit a compression target. Parent-bound delta capsules carry new counterexamples or dependency
182 facts while retaining the invariant core. Stable prefixes and provider cache reuse remain bound to
183 model/tokenizer/policy identities. [D1; ASEH, PCSM] A model request names the residual question,
184 admitted evidence, missing information, budget, expected response schema, and independent accep-
185 tance test. Failed candidates expand only implicated source or contracts when possible. This adapts

186 abstraction refinement to context selection without claiming that every context heuristic is a sound
 187 program abstraction.

188 5.3 Compile repeated successful work into bounded operators

189 ProcedureCegis already implements a bounded synthesis state machine. Its source order is existing
 190 verified procedure, built-in template, anti-unified pattern, declarative/constraint candidate, model
 191 sketch, and human candidate. Candidate and counterexample-set identities prevent repeated evaluation
 192 of the same pair. The synthesizer neither executes nor certifies nor promotes its output. [A2] The
 193 incoming memory method mines independently admitted trajectories, preserves failed episodes,
 194 infers applicability conditions, and validates anti-unified procedures on held-out families. Each
 195 procedure has typed parameters, effects, branch/loop bounds, residual holes, rollback, and required
 196 validation. Execution must recheck current applicability and authority. Learned call-target, event-
 197 delta, inverse-trace, and repair-delta models rank constrained candidates or abstain; their predicted
 198 transitions become accepted state only after observation or verification. [W2, W4] Architecture
 199 refactoring applies the same loop to module boundaries and state ownership: SCC-based candidates,
 200 compatibility contracts, source-preserving edits, differential traces, tests/proofs, and rollback. A
 201 smaller dependency graph, fewer files, or lower context burden only ranks candidates; it does not
 202 establish equivalence. [W4]

203 6 Test certificates and sound conditions for reuse

204 6.1 Specify exactly what is certified

205 A test certificate must name its statement, source/dependency roots, runner and environment, fixture
 206 inputs, result semantics, and checker/trust profile. “Verified” is not a single universal evidence class.
 207 A manifest can be complete while its child observations depend on a trusted runner; a circuit can
 208 correctly verify signatures while proving nothing about honest execution by their signer. [K1, A5]
 209 The source-present Groth16 adapter is gated by explicit opt-in and a compiled Rust backend, with
 210 BN254 and versioned circuit references. Its inspected derivation helper accepts only propositional
 211 atoms, facts, and one-antecedent implications, using forward chaining. This is not the full temporal/
 212 deontic system and not a CPython/pytest execution model. Versioned wrapper code, setup availability,
 213 and attestation metadata must not be mistaken for a qualified circuit/key ceremony or measured
 214 prover performance. [N9–N10]

215 Table 3: Zero-knowledge can hide witnesses; it does not enlarge the meaning of the proved relation.

216 Evidence class	Established relation	Remaining assumption or limit
217 Integrity / membership	These exact bytes or leaves belong to	Correctness and execution are not
218	this commitment.	implied.
219 Signed runner	A named authorized signer asserts the	Runner integrity and observation
220 observation	bound result.	remain trusted.
221 Formal proof	The encoded property follows in the	Translation adequacy, premises
222	declared logic/profile.	and checker TCB.
223 Aggregation / seal	Declared child checks and manifest/	No stronger meaning than the
224	transition rules hold.	verified child relations.
225 Direct execution proof	The specified machine/program	Actual modeled runtime, I/O
226	produces the specified result.	oracles, inputs and bounds.

227 6.2 Fixture-aware, phase-aware applicability

228 The PCTDD workstream separates collection seed, fixture-definition closure, materialized fixture-
 229 instance commitment, pre-call execution key, and post-execution setup/call/teardown evidence. It
 230 binds parametrization, plugins/hooks, confest, autouse fixtures, DI adapters, external snapshots,
 231 clocks/RNGs, interpreter, policy, and relevant hardware. Arbitrary Python object serialization is
 232 not a sound dependency model. Setup/call reuse may not silently skip required resource effects or
 233 teardown; a teardown failure prevents a whole-item pass. Opaque dependencies force execution.
 234 [W3]

235 6.3 Exact reuse and cross-tree reuse are distinct

236 A full-tree cache key changes on an unrelated edit. Broader reuse therefore needs a current dependency
 237 projection and a noninterference/applicability witness, not just removal of the tree from a key. If
 238 a deterministic observation under profile θ is a function $f\theta$ of a complete projection πD , then
 239 $\pi D(s) = \pi D(s')$ implies equal observations. This conditional argument is elementary; proving that πD
 240 includes all relevant inputs and effects is the hard obligation. Runtime traces alone cannot show that
 241 no unobserved dependency exists. [W3; Appendix I]

242 7 Incremental proof state and durable acceptance

243 7.1 Completeness is independent of membership

244 Verification is partitioned into source-addressed proof units with stable logical identities and complete
 245 applicability/trust keys. A requirement manifest declares every mandatory unit; a proof forest binds
 246 the evidence for those units in deterministic categories. Merkle membership proves inclusion, not
 247 that the collector included all required tests. Selection completeness and authorized removal of
 248 obligations are separately checked. Delta seals name the accepted parent and exact reused, replaced,
 249 added and removed units; uncertainty forces broader verification or a full checkpoint. [A5, K1]

250 7.2 Prepared artifacts cannot publish themselves

251 The durable store persists canonical blocks and candidate evidence; the supervisor runtime decides
 252 whether evidence satisfies current obligations. The small persistence adapter and actual pointer CAS
 253 are separate surfaces. Current-seal publication checks repository/branch identity, expected parent and
 254 generation. Local locks, atomic filesystem publication, and WAL recovery do not constitute global
 255 distributed consensus or a transaction spanning Git, the task database, and remote services. Crash
 256 recovery must explicitly reconcile those boundaries. [K1–K4]

257 candidate state and complete requirement manifest

258 → parallel prepare: canonicalize / hash / verify / build leaves

259 → recheck exact parent, scope, evidence and completeness

260 → persist children before referencing manifests

261 → one expected-parent publication barrier

262 → reconcile task disposition and publish accepted transition

263 Parallel canonicalization, hashing, independent proof checks, and Merkle levels are staged PCTDD
 264 optimizations. They must preserve sequential bytes, CID profiles, category ordering, roots, and
 265 dispositions. Chunk-manifest identities are distinct from a raw-file SHA-256 digest. Hash memos
 266 need immutable content/profile bindings; a path or modification time alone is insufficient. Required
 267 proof-pending states may continue local preparation but remain non-mergeable until mandatory
 268 evidence is admitted. [W3]

269 7.3 Acceptance invariant and remaining trusted base

$$\begin{aligned} \text{Publish}(s') \Rightarrow & \text{CurrentParent}(s) \wedge \text{Authorized}(a, s) \\ & \wedge \text{Scoped}(a) \wedge \text{CompleteManifest}(q, s') \\ & \wedge \text{EvidenceAdmitted}(q, s') \wedge \text{CAS}(s, s'). \end{aligned} \quad (2)$$

270 This is a design invariant to qualify on the actual call chain, not a theorem that all current code
 271 enforces it. A conditional preservation argument is possible: if the initial accepted state meets the
 272 selected artifact policy, every admitted transition checks that policy on exact inputs, and publication
 273 is exclusive, induction preserves that policy along the accepted lineage. It says nothing stronger than
 274 the checked policy and does not prove all physical side effects exactly once. Appendix I names the
 275 assumptions and proof obligations.

276 7.4 Multi-agent and world-state extension

277 Sibling supervisors exchange typed requests, observations, and receipts, not writable access to
 278 each other’s state. Mergeable observational indexes can nominate dependencies; exclusive claims,
 279 fences, policy, publication, and completion remain transactional. Accepted world roots can compose
 280 semantic, program-trace, logical-dataset, procedure, proof, and environment roots. Cyclic logical
 281 graphs use stable IDs and snapshot manifests rather than literal cyclic content links. Required-mode
 282 operation must exhibit pre-state consumption, accepted transition evidence, and post-state publication.
 283 [W1–W2; DOEP]

284 8 Evaluation: useful progress, preservation, and cost

285 8.1 Primary comparison and independent oracles

286 Freeze a task population, model revisions, exact repository forest, tools, policies, budgets, and
 287 acceptance criteria before running paired conditions. The core A/B comparison changes only raw/
 288 lexical versus semantic context; later arms add routing, incremental verification/reuse, and governed
 289 lifecycle. These combined arms measure package effects, not the isolated contribution of each
 290 mechanism. Orthogonal ablations must isolate translation checking, symbolic routing, context
 291 selection, proof reuse, parallel preparation, and procedures. [ASEH; PCPR]

292 8.2 Populations and controls

293 Use exact pre-fix historical tasks, held-out task/repository families, and controlled adversarial cases.
 294 Hide target patches and acceptance oracles from proposal generation; generated tests may supplement
 295 but not silently replace the oracle. Include valid authorized actions as well as invalid ones. Seed
 296 lost modalities, inconsistent assumptions, removed tests, missing fixture inputs, stale maps/keys,
 297 changed verification keys, unknown provider outcomes, and crashes around publication. Keep failures,
 298 timeouts, abstentions and unavailable routes visible. Measure successful accepted tasks per fixed
 299 budget, false acceptance and false denial, conditional solve rate after dispatch, time-to-first-valid-task,
 300 event-to-replan latency, task churn, manual repair,

301 Table 4: Pair safety endpoints with useful-progress endpoints. Zero unsafe actions is trivial for a
 302 deny-all system.

303 Study	Matched comparison	Primary correctness / progress outcome
304 Context	Raw/retrieved source vs semantic packs; same model/checks.	Completed accepted tasks; missed regressions; tokens and fallback.
305 Symbolic control	Model-first vs obligation-driven routing.	Correct deterministic closures and authorized residual dispatch, not just blocked calls.
306 Repair / synthesis	Existing template/CEGIS vs model proposal under same scope.	Independent hidden/full validation; specification preservation; rejected candidates.
307 Test / proof reuse	Cold execution vs admitted reuse; dependency-changing attacks.	False reuse; cold-oracle agreement; fixture lifecycle; coverage.
308 Sealing / state	Sequential vs parallel; crashes and stale writers.	Identical roots/dispositions, accepted lineage, recovery and useful continuation.

319 Table 5: Repository-reported preliminary evidence; not experiments rerun during this paper revision.
 320 [A4–A5]

321 Existing report	Bounded observation	Cannot establish
322 Semantic context harness	40 controlled tasks; 58.90% median estimated context reduction; no model receipts.	Live inference savings, general repair accuracy, or useful provider dispatch.
323 Incremental sealer	40 transitions; 43.6161% reuse; 43.0539% estimated compute reduction.	Measured prover speedup; GPU cost; proof that arbitrary tests ran.
324 Current source inspection	Typed state, routing, formalization, bounded synthesis, and storage mechanisms.	A single fully integrated deployment, unpushed worktree behavior, or passing runtime qualification.

and each evidence class. Instrument actual provider boundaries: a gate that makes no model call internally does not establish that the workflow saved a model call. Warm-cache and cold-start conditions require separate results.

8.3 Economic and statistical reporting

Report actual input/output/cached tokens, model calls, provider charges, CPU/GPU time, memory, storage, fixture setup/call/teardown, proof generation/checking, recovery, and human time. Currency conversions require a stated tariff; unavailable values are not zero. Measure end-to-end elapsed time directly rather than summing overlapping parallel durations. Report amortization and the break-even number of repeated tasks. Use paired differences, medians, confidence intervals, failure counts with denominators, and stratification by dynamic-feature and task class. A finite zero-failure count is not universal safety. Preregister a quality/noninferiority margin and minimum useful completion threshold; efficiency promotion requires both. The final results and exact tested forest remain [TO BE FILLED].

9 Evidence limits, related work, and conclusion

9.1 What the existing artifacts establish

The reviewed standalone repository heads and the consumer’s dependency pins differ. A final claim must bind the actual loaded source forest, including overlays, native binaries, solver/circuit profiles and active rollout settings. Source-present intent authorization and a Groth16 wrapper must not be described as always-on enforcement or a validated universal prover. The preceding integration findings belong in the author audit until resolved or reported as limitations.

9.2 Relationship to established methods

Proof-carrying code attaches checkable policy evidence to untrusted code [1]; abstract interpretation and CEGAR reason over approximations and refine them with counterexamples [2, 3]. Translation validation checks a produced transformation, and CEGIS searches bounded candidates against counterexamples [4, 5]. Equality saturation represents alternatives under rewrite relations [6]. These are foundations, not newly invented algorithms in this work. The intended difference is composition at the agent’s operational boundary: persistent semantic state, explicit obligations, residual model authorization, phase-aware test evidence, and accepted-transition publication. Build-system research separates scheduling from rebuilding [7]; our reuse design additionally exposes statement, fixture, trust and authority conditions. Coding agents and test-generation benchmarks [8, 11], premise-retrieval and hammer systems [9, 10], and prompt compression [12] provide relevant baselines. We do not claim that this is the first neurosymbolic agent or that combining these techniques automatically improves performance.

9.3 Limitations and accountable self-improvement

Python reflection, native code, external state, concurrency, incomplete specifications, and unsound translations limit what can be proved. A sound checker cannot repair an incorrectly formalized requirement. Default-deny behavior can mask broken positive paths. A self-improving supervisor must not edit its own protected acceptance specification, keys, validators or promotion policy under ordinary task authority. Every learned update or compiled procedure requires an independent holdout and rollback gate. Important model use in implementation, formalization, candidate generation, experiments and manuscript preparation must be disclosed accurately. The final claims remain conditional on the frozen evidence and tested deployment. Our thesis is that persistent obligations can displace repeated neural reasoning and validation while preserving a declared acceptance standard; the paired evaluation must determine whether the total benefit exceeds the added analysis and proof cost.

References

- [1] George C. Necula. Proof-carrying code. POPL, 1997, pp. 106–119. DOI: 10.1145/263699.263712.
- [2] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. POPL, 1977, pp. 238–252.
- [3] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. CAV, 2000, pp. 154–169. DOI: 10.1007/10722167_15.
- [4] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. TACAS, 1998, pp. 151–166.
- [5] Armando Solar-Lezama. Program Synthesis by Sketching. PhD dissertation, University of California, Berkeley, 2008.
- [6] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. POPL, 2021. DOI: 10.1145/3434304.
- [7] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. ICFP, 2018. Expanded as Build systems à la carte: theory and practice, Journal of Functional Programming 30, 2020.
- [8] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 37, 2024.
- [9] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Animashree Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. NeurIPS 36, 2023.
- [10] Albert Qiaochu Jiang, Wenda Li, Szymon Tworkowski, Konrad Czechowski, Tomasz Odrzygóźdź, Piotr Miłoś, Yuhuai Wu, and Mateja Jamnik. Thor: Wielding hammers to integrate language models and automated theorem provers. NeurIPS 35, 2022.
- [11] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-Bench: Testing and validating real-world bug-fixes with code agents. NeurIPS 37, 2024.
- [12] Zhuoshi Pan et al. LLMingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. Findings of ACL, 2024, pp. 963–981. DOI: 10.18653/v1/2024.findings-acl.57.

406 A One complete state to repair to certificate cycle

407 Scenario: a repository operation writes a record before checking a caller's permitted scope. The
408 authorized requirement is that a denied request produces no persistent write. R0, P0 and T0 below
409 are illustrative labels, not recorded benchmark CIDs. The example exercises the planned integration
410 rather than claiming an observed end-to-end run.

411 Table 6: Illustrative state-to-repair-to-certificate cycle; labels are
412 schematic.

413 Step	Artifact and acceptance condition
414 1. Capture state	Freeze source tree R0, recursive dependencies, 415 environment E0, policy P0 and semantic root S0. Extract 416 the call/effect edges for authorize and write_record; keep 417 any opaque store behavior visible.
418 2. State the obligation	O0: denied(request, P0) implies persistent_state_after = 419 persistent_state_before for the declared observation 420 profile. Confirm this is an authorized requirement; do not 421 infer it only from existing tests.
422 3. Obtain a counterexample	A denied request under fixture instance F0 produces a 423 write. Bind the failing call trace, fixture-definition and 424 instance commitments, runner profile, setup/call/ 425 teardown outcomes and exact source.
426 4. Plan a bounded repair	Localize the write-before-check ordering. Admit a plan 427 that changes only this operation and its justified 428 dependencies. The candidate must preserve authorized 429 writes, error behavior, finalization and public API 430 semantics.
431 5. Choose a reasoning route	Try an existing checked guard-before-effect template. 432 Use abstract effects or an encoded transition property to 433 discharge what the model supports. If ordering or 434 callbacks remain ambiguous, expand exact source or 435 request a bounded model residual-not an unrestricted 436 rewrite.
437 6. Validate the candidate	Apply to a fenced disposable worktree R1, rescan to S1 438 and invalidate the affected contracts/tests/proof units. 439 Execute relevant tests and full fallback where required. 440 A fake store test does not prove production database 441 semantics.
442 7. Reuse narrowly	A proof unit outside the complete affected closure may 443 be reused only after current binding and evidence checks. 444 A policy or fixture change invalidates every dependent 445 unit even if the test function text is unchanged.
446 8. Seal and publish	Admit the current requirement manifest, persist verified 447 children and a parent-bound delta seal, then 448 compare-and-swap the expected pointer/generation. 449 Reconcile the operational task only after matching 450 publication and validation receipts.
451 9. Learn without self-approval	Store the accepted transition and rejected candidates. 452 Repeated independently validated instances may 453 nominate a guard-before-effect procedure; synthesis, 454 certification and promotion remain separate.

455 A.1 Counterfactual checks

456 Change F0, P0, the interpreter, a plugin, teardown behavior or an external snapshot without editing
457 the test body: the key must change or reuse must be rejected. Deliver a duplicate event or late worker
458 completion: the current state must not regress. Crash after immutable persistence but before pointer

459 CAS: recovery must not invent completion. Supply an empty manifest while O0 is required: reject
460 incompleteness. The example's proof claim is limited to the selected model and observations. A
461 successful unit test, a checked transition formula and an attested runner result remain different
462 artifacts with different trust assumptions.

463 B A typed evidence model and verification obligations

464 B.1 Evidence record

465 Evidence = (claim, subject_roots, statement_or_observation, scope,

assumptions, dependency_roots, translation_receipts,
 checker_profile, checker_result, environment, policy,
 producer_or_signer, invalidation_bindings, provenance)
 $\text{Admit}(e, q) = \text{Integrity}(e) \wedge \text{SchemaValid}(e) \wedge \text{Applicable}(e, q)$
 $\wedge \text{RequiredCheckPasses}(e, q) \wedge \text{PolicyAllows}(e, q)$

This is a proposed unifying notation, not a replacement for existing schema authorities. Integrity checks exact bytes; applicability checks freshness and scope; required checking evaluates the selected statement; policy checks execution/acceptance authority. The conjunction cannot be satisfied by a model’s confidence score or a single untyped success flag. A verified proof may originate from an untrusted model: the checking rule, not the proposal’s reputation, establishes the new claim.

Table 7: Formal mechanisms and their required evidence.

Mechanism	Representation and required limit
Abstract interpretation	Bind transfer semantics, abstract domain and widening/narrowing strategy. Unsupported operations over-approximate or become opaque. Do not call the entire Python front end sound solely because one abstract domain is well defined. [D3]
Assume-guarantee	Bind producer/consumer clauses, source map, theory, invariants and normal/exceptional observations. Check assumption consistency and cyclic closure; expose uncovered prose clauses. [D4]
Incremental SMT	Bind full formula/assertion roots, declarations, theory, translator, solver configuration and replay manifest. The current wrapper rebuilds its solver; benchmark any future retained-state implementation separately. [D2]
Interpolation and refinement	Validate implication, contradiction and shared vocabulary; preserve unsupported/timeout outcomes. CEGAR refines abstractions, while CEGIS refines bounded candidates against accumulated counterexamples. [D5]
Tactician / Hammer / proof assistants	Premise and tactic predictions are candidates. Invoke the actual available backend and native reconstruction or checker required by the profile; no fake bridge, universal provider or cross-logic soundness assumption. [D7; W2, W4]
Translation / e-graph extraction	Freeze source/target languages, supported observations, rule trust and proof replay. Equi-satisfiability, refinement and observational equivalence are different claims. Opaque interface names are not implementation evidence. [D6]

B.2 Targeted source-level checks before the final freeze

Add tests for duplicate or colliding named SMT assertion/assumption identifiers, exact formula-root binding in receipts, stale replay fingerprints, unsupported terms and rejection of unvalidated results. Exercise the real caller admission path—not only a synthetic result constructor. These are review obligations, not a claim that a runtime exploit was demonstrated in this audit.

C Fixture definitions, injected instances and reuse eligibility

This appendix preserves the parallel sealing/TDD directive’s staged protocol. Earlier identity and trace infrastructure exists, but no new qualification of this entire protocol is asserted here. [P8; W3]

Table 8: Staged test identities and permitted reuse.

Stage	Required bindings	Decision allowed
0 · collection seed	Test node, parameters, marks, collection config, source/AST identity, static closure, plugins and adapters.	Candidate lookup only; not a passing outcome.
1 · fixture-definition closure	Fixture names, definitions, scopes, autouse/usefixtures, parameterization, transitive dependencies, confest, hooks, setup/teardown/finalizers, purity/effect policy.	Whole-item pre-setup reuse only under a qualified closed policy.

524	2 · fixture-instance	Resolved parameter values and	Post-setup eligibility; opaque or
525	commitment	reviewed injected commitments;	mutable unbound values force
526		filesystem/database snapshot; service	execution.
527		image and response corpus;	
528		transaction/lease, clock/RNG and	
529		setup outcome.	
530	3 · execution key	Test/source/fixture closure, instance	Re-admit a prior receipt or
531		roots, runtime and dependency lock,	execute the call. No optimistic
532		toolchain, relevant hardware, effect/	test pass.
533		network policy, trace completeness,	
534		statement/circuit/key profile.	
535	4 · phase receipt	Setup, call and teardown outcomes,	Fresh observation and evidence
536		failures, effects, observed dependency	for later reuse.
537		trace, runner identity, final acceptance	
538		and signature/proof relation.	

539 C.1 Reviewed dependency-injection adapters

540 Admit canonical primitives, bounded typed records, exact file/resource CIDs, versioned configuration
541 allowlists, immutable database snapshots, reviewed service fakes or committed response corpora
542 through explicit adapters. Do not serialize arbitrary callbacks, sockets, secrets, unrestricted object
543 graphs or live databases as if they were complete immutable inputs. Bind adapter version, privacy
544 policy and any absence or non-determinism. [W3]

545 C.2 Preserve lifecycle effects

546 Separate reuse of fixture definitions, fixture instances and test-call results. Module/session-scoped
547 resources, parametrized instances and xdist worker ownership require scope-aware scheduling. If
548 setup ran, required teardown/finalizers still run even when a call result is reusable. Teardown failure
549 prevents a whole-item pass. Scheduling must preserve resource isolation and observed ordering
550 requirements, not simply distribute tests with matching names. [W3] A trace records what was
551 observed on one execution. It does not prove that a branch not taken could not access another
552 dependency. Exact reuse across states needs a closed behavioral boundary;

553 otherwise widen the closure and execute. Keep cold executions and mutation cohorts to test the reuse
554 policy independently.

555 C.3 Proof batches, parallel preparation and the inner loop

556 C.3.1 Certificate and batch statement

557 A batch statement binds the test inventory/requirement manifest, child execution keys and admitted
558 outcomes, count, ordering, duplicate policy and aggregate root. A cryptographic profile must bind its
559 public inputs, any private commitments, circuit or program, verification key, setup origin, verifier
560 version and intended relation. A proof of signature verification still relies on the signer's execution
561 claim; it is not direct proof of the runner. Zero-knowledge is justified only by a defined privacy
562 need and a reviewed statement. [W3] Reject unknown circuits or key origins, test-only keys on
563 production allowlists, silent key generation, simulated required units, and outer metadata not bound
564 to the verified statement. Keep proving keys and witness material outside public proof-store APIs. If
565 recursive verification is unavailable, represent a manifest of individually verified leaves rather than
566 inventing a recursive proof. [K1; W3]

567 C.3.2 Parallelism that preserves identity

568 Table 9: Parallel preparation and serial publication obligations.

569	Preparation job	Required safety property
570	Read / parse / canonicalize	Use an exact immutable input or verified overlay.
571		Exclude symlink escapes, unstable reads and mutable
572		pathname assumptions; canonicalize once under a fixed
573		profile.
574	Hash memo	Key exact content and relevant canonicalization/codec/
575		hash/source-mode policy. Reverify candidates; mtime or
576		a fast digest alone is not the final SHA/CID authority.
577	Hash / verify workers	Parallelize independent records/files/proof checks with
578		bounded CPU, memory, process/GPU usage,

579 cancellation and deadlines.
580 Merkle construction Sort leaves, build categories and same-level nodes
581 concurrently, preserve normative order. Memoize
582 unchanged verified branches and recompute paths from
583 changed leaves.
584 Serial barrier Recheck completeness and current identities, persist
585 children, then use the accepted expected parent/
586 generation. Worker completion order cannot change
587 bytes or disposition.

588 C.3.3 Development-loop states

589 The fast loop combines source capture, semantic invalidation, selection, cache-candidate lookup,
590 fixture-aware execution, candidate repair and certificate production. Proof generation can proceed
591 asynchronously inside the product only when the intermediate state remains explicitly non-mergeable.
592 Queue starvation, cancellation, proof failure, missing keys or unknown execution must preserve honest
593 status. [W3] The directive’s rollout is bootstrap → shadow_hash → shadow_reuse → shadow_proof
594 → protected → required. These gates are not interchangeable with world-model shadow-read
595 semantics. Measure call/setup/teardown time, proof latency, batch verification, hash/serialization
596 throughput, cache corruption rejection, serial barrier time and total cost. Performance targets are
597 acceptance goals—not results. [W3]

598 D Semantic objects, projection indexes and logical datasets

599 D.1 Semantic objects and relations

600 Wrap existing domain-state CIDs in adjacent versioned semantic-object envelopes containing domain,
601 schema, canonicalization profile, provenance, source CIDs, optional declared policy salt and evidence
602 class. Do not alter the existing source-semantic root to include embeddings, prompts, actors, leases,
603 wall time or transition history. Pure semantic deltas remain distinct from operational execution
604 records. [W1, W2] Relations are scoped claims such as equivalent, refines, entails, contradicts,
605 compatible_with and supersedes. Each binds left/right objects, scope, assumptions, environment
606 and theory/policy, proof/ evidence references, limitations and invalidation dependencies. “Similar”
607 belongs to a retrieval projection, not the authoritative semantic relation vocabulary. Exact CID
608 equality establishes byte identity, not universal semantic equivalence. [W1]

609 D.2 Hybrid retrieval without authority leakage

610 Projection records bind their semantic object, projection kind, model, tokenizer, preprocessor, dimen-
611 sion, metric, normalization, vector identity and privacy policy. Rebuildable index manifests bind
612 generations and model/projection compatibility. Lexical, vector, LSH/Hamming, graph or neural
613 search may nominate candidates. The resolver must retrieve exact object bytes, verify current root
614 membership and recheck freshness, environment, obligations and policy before use. A stale vector
615 or approximate neighbor never substitutes for an exact subject. [W1, W2] The graph-to-graph
616 or typed-attention TAGSeq/DAGSeq idea remains an optional experimental projection over typed
617 program neighborhoods. It must be compared with exact lookup, rules, ordinary graph features and
618 embedding-only baselines. Canonical symbols and typed source maps remain authoritative even if
619 the learned projection changes. [W2]

620 D.3 Logical data versus physical layout

621 LogicalDatasetState and PhysicalDatasetLayout must not collapse into one identity. Under a fixed
622 logical schema, key/order policy and provenance, changing Parquet row groups, compression or shard
623 packaging should preserve logical content identity while changing layout identity. Set, multiset and
624 ordered semantics must be explicit; duplicate and key-conflict behavior cannot be inferred from file
625 order. [W1] Transformations bind exact inputs, transformation specification, environment/toolchain,
626 candidate outputs and validation evidence. Merges use a closed declared semantics—append, keyed
627 three-way, ordered, set/multiset, schema evolution or conflict. Distributed workers may prepare
628 shards and candidate layouts, but only accepted logical state and validated provenance may enter a
629 world root. Legacy or mock dataset managers cannot silently remain production semantic authorities.
630 [W1, W2]

631 D.4 VFS bridge

632 A successful canonical VFS/worktree mutation emits a durable post-commit receipt and semantic
633 outbox record. Rescanning, invalidation and acceptance occur through the existing semantic and
634 supervisor authorities. Filesystem success alone is not task success; recovery handles an outbox that
635 exists before semantic acceptance, and all side effects remain attributable to exact operations. [W1,
636 W2]

637 D.5 Frames, events, transitions and learned continuation models

638 D.5.1 State schema and abstraction profile

639 ProgramStateSnapshot:

640 repository/environment/program-graph roots; source location; stack;

641 selected locals or shapes; abstract heap/alias summary; path condition;

642 exception state; obligations; capabilities/effects; external observations;

643 capture profile; abstraction profile; confidence and limitations

644 ProgramTransition:

645 before-state; action; event; after-state or delta; conditions;

646 observation/validation evidence; accepted/rejected/candidate status

647 Profiles declare stack depth, value precision, heap slicing, allocation identity, timestamp treatment,
648 exception handling and the observations preserved or erased. Absence is typed: a trace adapter that
649 cannot capture an external service is not a complete program-state observer. Logical execution graphs
650 may cycle without creating literal cyclic content-addressed blocks. [W2]

651 D.5.2 Three distinct inference problems

652 Table 10: Program-prediction tasks and candidate semantics.

653 Task	Candidate semantics and validation
654 Next call	Use lexical scope, imports/exports, nominal/structural
655	types, alias/points-to information, inheritance, dispatch
656	tables, callbacks, plugins and observed traces to build a
657	conservative set. Rank existing targets; keep
658	unknown-dynamic probability and abstention.
659 Next event / abstract state delta	Predict call/return/raise/handle/branch/await/yield/
660	resume/I/O/mutation/contract/check events with a
661	declared state abstraction. Validate by replay, execution,
662	contracts or the appropriate formal backend.
663 Inverse trace	Return multiple predecessor hypotheses, missing calls/
664	inputs and replay recipes from a partial failure.
665	Non-invertibility and observational ambiguity are
666	expected; do not train it as a unique true reversal.

667 Symbolic pruning may exclude a candidate only through an admitted type, path, alias, effect, temporal,
668 resource, authority, exception or contract constraint. Uncertainty is not exclusion. A repair-delta
669 model proposes bounded changes to canonical nodes/edges; an exact resolver and source-preserving
670 renderer still need independently validated source correspondence. [W2]

671 D.5.3 Dataset and promotion protocol

672 Freeze capture/abstraction profiles, provenance, source-family splits, candidate-set construction and
673 targets before evaluation. Separate observed from synthetic/static-derived examples and distinguish
674 valid alternatives from wrong predictions. Report candidate recall before ranking, top-k, calibration,
675 abstention, unsupported cases, replay validity and downstream repair value. Model scores do not
676 become facts or required certificate evidence. [W2] The predecessor proof-grounded learning
677 campaign and a successor's uncompleted work cannot share a positive-result label. Preserve the
678 earlier reported no-go until a distinct frozen learned artifact and independent evaluation establish a
679 new claim. This review does not establish new learned weights, a tokenizer or a universal interlingua.
680 [P6, P11; W2]

681 **E Behavior-preserving remodularization, not cosmetic compression**

682 **E.1 Model the observations to preserve**

683 Track function, class, block and module identity at multiple levels: exact source/CST, semantic imple-
684 mentation, binding, contract, effects, mutable state, dependencies, public interface and compatibility.
685 Formatting, relocation, renaming, aliasing and semantic change need different dispositions. Public
686 behavior includes return values, exceptions, mutation, initialization order, import effects, serialization/
687 pickle names, introspection, plugin discovery, cancellation, timeouts, resources and documented API
688 surfaces—not just existing happy-path tests. [A3; W4]

689 **E.2 Analyze before partitioning**

690 Build a typed architecture graph of calls, reads/writes, authority owners, providers, interfaces, receipt
691 producers, stores, tests and proofs. Identify SCCs, cycles, import-time ordering and shared state. A
692 partition candidate must preserve a coherent state owner and an explicit boundary contract; do not
693 break a dependency cycle by hiding it behind opaque callbacks or duplicated state. Similarity and
694 semantic-entropy scores nominate candidates but prove neither duplication nor dead code. [A3; W4]

695 **E.3 Bounded transformation and validation**

696 architecture finding → partition candidate → boundary contracts

697 → scoped operator + reversible migration plan

698 → isolated worktree + CST/AST-aware transform

699 → translation / differential / effect / public-surface checks

700 → selected and full-fallback tests + supported proofs

701 → independent admission + exact rollback + updated state

702 Reuse a closed operator grammar: extract module/interface/pure function, move state to its owner,
703 introduce typed adapters or dependency inversion, quarantine legacy/simulation paths, replace
704 ambiguous outcomes, deprecate a public symbol or remove confirmed dead code after the relevant
705 gate. Operators bind preconditions, scope, effects, migration, consumer coverage and rollback.
706 Unknown dynamic reachability is not proof of deadness. [A3; W4] Use property-based, metamorphic,
707 mutation and differential executions over declared observables, plus type/effect checks and supported
708 solver or native proof obligations. Tactician/Hammer routes can try to discharge obligations; bounded
709 synthesis or a small model may propose a residual. A test suite cannot alone establish universal
710 observational equivalence of arbitrary Python. Contradictory authoritative premises require conflict
711 resolution, not proof by inconsistency to authorize a rewrite. [W4]

712 **E.4 Fixed-point control and memory**

713 Persist accepted and rejected partitions, failed applicability, counterexamples and migration costs.
714 Recompute only affected graph regions; stop when no admissible beneficial candidate remains or
715 a budget is exhausted. Public API, mutable-state, security, legal, payment, key, release-authority
716 and wire-protocol changes retain their human-approval ceilings. Learned benefit or lower context
717 cost cannot raise those ceilings. [A3; W4] The public architecture checkpoint establishes inventory
718 and ArchitectureIR foundations. The broader SCC/partition/CST/remodularization program remains
719 separately tracked against the author's local worktree. [B1; W4]

720 **F Rollout, event federation and verified self-hosting**

721 **F.1 Reconcile predecessor programs**

722 The semantic-world and program-world directives explicitly require extending an accepted predeces-
723 sor or creating a sealed successor—not two stores, scanners, schedulers or root authorities. Apply the
724 same rule to existing procedure, architecture, proof reuse, sealer, governor and meta-controller

725 campaigns. Read the live owner-controlled board and dependency seals before adding tasks; old plan
726 status text does not override current code or active local work. [W1–W4]

727 **F.2 Distinct rollout state machines**

728 Table 11: Distinct rollout state machines.

729 Program	Meaning of progression
730 World / program-world	Bootstrap → shadow_write → shadow_read → guarded → required. In the program-world directive, shadow_ 731 read computes hypothetical decisions without 732 influencing behavior. Guarded allows only bounded 733 consumption under independent checks. Required makes 734 pre-root consumption and post-root acceptance part of 735 task completion.
736 Parallel sealing / TDD	Bootstrap → shadow_hash → shadow_reuse → shadow_ 737 proof → protected → required. Require sequential parity 738 and zero unsafe divergence on the fixed adversarial 739 cohort before protective gates can depend on the new 740 path.
741 Refactor / procedures	Separate candidate generation, independent qualification, 742 shadow use and policy-authorized promotion. No worker 743 changes its own assurance threshold or rollout mode. 744

745 **F.3 Mandatory per-task receipts in required mode**

746 A task consumes an exact source forest, semantic/program state, obligation set, route decision and
747 context plan. It produces candidate artifacts, observations, current validation, admitted or rejected
748 transition, and a matching post-root/completion record. A deployment that writes optional semantic
749 metadata while continuing to decide from raw prompts has not demonstrated required self-hosting.
750 [W2, W3]

751 **F.4 Concurrency and persistence**

752 The operational database is accessed through the current typed owner boundary; event and outbox
753 records, revisions, claims and fences must agree. Delivery can be at least once while logical operations
754 are idempotent. Causal graphs and affected-frontier wakeups avoid duplicate reconstruction, but
755 retrieval projections and optional history stores cannot create readiness, leases or acceptance. [P3;
756 W2] CRDT-like merging can be useful for observational knowledge with explicit conflict semantics.
757 It must not merge exclusive leases, authorization, current roots, irreversible effects or completion by
758 optimistic set union. Speculative agents use isolated branches and candidate proof/premise prefetch;
759 only independently admitted outputs cross the serial publication boundary. Remote storage and
760 libp2p/MCP++ transport require their own authentication/capability and fault tests. [W1, W2]

761 **F.5 Typed service parity**

762 Python, CLI and MCP are projections of the same operation contracts and authority rules. An
763 envelope identity is not the semantic payload identity. Delegated capabilities authorize an operation
764 but do not establish its mathematical correctness. Keep secrets and raw private traces outside prompt,
765 receipt and public-storage surfaces; ordinary imports must not launch daemons or installers. [D1;
766 W1–W4]

767 **G Experiments and acceptance gates for the landing work**

768 Table 12: Experimental cohorts and required fault checks.

769 Cohort / ablation	Required measurements and failure checks
770 State and selection	Cold/incremental byte-root parity; deletion/rename/ 771 config/fixture/native/dynamic changes; selected-vs-full 772 regressions; ambiguity and raw-source fallback. Empty 773 denominators stay not-applicable.
774 Planning and repair	Same tasks, models, budgets and acceptance criteria. 775 Count calls by route, deterministic closures, proof holes, 776 repaired/unsolved/escalated tasks, duplicate retries and 777 human interventions.

778	Certificate reuse	Cold oracle executions with changed fixtures, instances, finalizers, plugins, policies, external snapshots and runner versions. Report each attempted false reuse, not only aggregate hit rate.
782	Parallel sealer	Equal bytes/roots/dispositions at multiple worker counts; malformed blocks, cache corruption, interrupted writes and stale-parent races. Separate prepare, verify, persist, CAS, CPU/GPU and memory costs.
786	Procedures / world models	Held-out repository-family and time-based splits; candidate recall, applicability, replay, calibration/abstention, negative memory and transfer. Synthesis/training cannot inspect held-out acceptance outputs.
790	Architecture refactoring	Frozen before/after tasks; API/import/exception/effect/serialization behavior; dependency cone and context burden; migration, validation, rollback and subsequent maintenance cost.
794	Recovery and federation	Duplicate/reordered events, worker crashes, expiry/fencing, unknown external effects, store unavailable, network partition, cross-repository dirty overlays and conflicting gitlinks.

798 G.1 Cost and uncertainty

799 T_{total} = elapsed wall time from task admission to terminal disposition
800 Stage report = capture, compile, retrieve, model, setup, test, teardown,
801 prove, verify, persist, retry, recover and human waits
802 Money_{total} = billed provider cost + measured infrastructure allocation
803 Resource report = CPU time, GPU time, peak memory, bytes, storage growth
804 Overlapping parallel stage times are not additive wall time. Do not add seconds, tokens and dollars as
805 if they were one physical quantity. A policy may optimize a declared weighted utility, but the paper
806 must also report the separate observable units. Amortize cold indexing, proof setup, training and
807 procedure compilation over an explicit number of tasks; include break-even behavior and workloads
808 where the method is slower.

809 G.2 Hard gates and honest dispositions

810 Require no observed unsafe admission, stale publication, forged certificate acceptance or false reuse in
811 the frozen adversarial suite, while reporting its finite size and uncertainty. Quality must not be traded
812 away to meet a speed target. An unavailable backend, an incomplete proof or baseline-compatible
813 failing test remains visible. A non-promotion can be the correct campaign result. The final anonymous
814 bundle needs a source forest, install/lock and capability manifests, frozen task population/splits,
815 model settings, test/prover commands, raw receipts, analysis code, failure
816 taxonomy, trusted-base manifest and replay instructions. A reviewer must be able to regenerate each
817 table without private author state.

818 H Early supervisor programs and their scientific role

819 The chat contains more than the four saved world/TDD/refactoring directives. The earlier programs
820 below are retained explicitly. They are reconciled with existing implementation owners rather than
821 treated as commands to launch duplicate boards. A handoff prompt specifies intended work; it is not
822 evidence that a supervisor was launched or that a worktree landed.

823 Table 13: Earlier supervisor programs and their qualification requirements.

824	Program	Method preserved	Qualification needed
825	ASEH · efficiency	Paired end-to-end telemetry;	Actual provider calls, success and
826	and state hardening	deterministic-first routing; shared	cost at matched budgets; event/
827		contexts; one state-transition	recovery and warm/cold
828		authority; removal of duplicate	baselines.
829		writable paths.	
830	DOEP · direct	High-level idea → typed objective →	Valid input reaches useful tasks;
831	objective and event	goal/task DAG; event-driven	changed state invalidates affected

832	planning	PlanDelta and bounded refill; sibling	work; no runaway objectives or
833		clients; nonconvergence and authority	queue-empty success.
834		controls.	
835	PCPR · platform	Truthful unavailable/simulation	One loaded integrated forest,
836	qualification	statuses, canonical path cutovers, real	active rollout, clean installation
837		backend qualification, immutable	and current evidence; not three
838		dependencies and release manifest.	unrelated main-head results.
839	PCSM · semantic	Canonical semantic IR versus local	Separate slicing/alias/linking
840	minification	projection; M1 source-linked edits,	ablations; exact target source;
841		M2 declared abstraction, M3 advisory	round-trip and stale-map tests; no
842		learning; tokenizer costs and CEGAR	universal AST linker inferred
843		expansion.	from a patch schema.

844 H.1 Dependency order, not a new campaign taxonomy

845 First qualify state and applicability, then the symbolic route and real positive model-dispatch path,
846 then repair and test-certificate admission. Only after those seams work should a combined run
847 attribute gains to automatic refill, minification, procedures or learned world state. Existing gates and
848 protected validation owners remain authoritative.

849 H.2 Independent status dimensions

850 location: main | branch | local_overlay | specification_only
851 implementation: present | partial | incompatible | missing
852 integration: called | adapter_only | optional | unknown
853 validation: not_run | reported | reproduced | failed | blocked
854 rollout: off | shadow | guarded | required
855 paper claim: exact mechanism + population + scope + limitations
856 A branch may already be an ancestor of main; an importable module may never be consumed; a
857 passing negative fixture may never exercise a valid path. Update each dimension independently. Do
858 not replace pending outcomes with the expected landing date.

859 H.3 World state, tests, and refactoring are parts of one loop

860 Table 14: World-state, test, and refactoring methods in the supervision
861 loop.

862	Directive	Mechanisms retained	Connection to supervision
863	W1 ·	Semantic objects and scoped relations;	Accepted pre/action/post
864	semantic-addressed	canonical profiles; projection/index	transitions become reusable state;
865	world	generations; exact resolution; logical	similarity can nominate but never
866		datasets vs layout; VFS outbox and	substitute for exact resolution.
867		world-root CAS.	
868	W2 · program world	Frames/stacks/heaps, capture/	Obligations and counterexamples
869		abstraction profiles, typed traces;	determine route, context, subgoals
870		static candidates and symbolic	and tasks; learned output stays
871		pruning; next-call, next-event,	candidate-tier until validated.
872		inverse-trace and graph-delta	
873		residuals.	
874	W3 · parallel sealing /	Hash memo and parallel preparation;	Avoid repeated work only when
875	TDD	deterministic forests;	current inputs, effects, evidence
876		fixture-definition/instance and DI	and publication checks permit it;
877		keys; whole-item vs call reuse; setup/	cold execution is the reference.
878		call/teardown; xdist; async	
879		certificates.	
880	W4 · semantic	Multi-level identities; authority/state	Repair/refactor waves preserve
881	refactoring	graphs; SCC partitions; boundary	observations under declared
882		contracts; CST-aware codemods;	profiles; verified episodes support
883		initialization, API, pickle and plugin	bounded procedures, negative
884		compatibility.	memory and fixed-point
885			reanalysis.

886 The four saved directives collectively specify 126 top-level sections, including operational and report-
887 ing requirements. Their previous complete heading index remains in the companion reconciliation
888 handoff; it has not been replaced by this method summary. The latest revision retains Appendices

889 A–G and adds exact source-level qualification issues rather than discarding the previously discussed
890 mechanisms.

891 **H.3.1 Self-hosting must be observable**

892 In shadow-write the system computes sidecar artifacts; in shadow-read it evaluates hypothetical
893 decisions without changing authority. Guarded consumption and required self-hosting must be
894 separately approved. In required mode each eligible task consumes a verified pre-root, current
895 obligations and route/context decision, then publishes observations, validation, accepted/rejected
896 transition and completion/post-root evidence. Workers may not modify rollout, keys, protected
897 specifications or their own acceptance validator.

898 **H.3.2 Learned representations remain bounded research**

899 Typed-attention/TAGSeq/DAGSeq-style models are optional comparisons after static, history, lexical
900 and embedding baselines. Program graphs may be cyclic; storage manifests are not. Predicting a next
901 event, recovering a predecessor trace and proposing a repair are different objectives with different
902 labels. Track calibration, abstention, static-candidate recall, leakage-safe repository-family/time
903 splits, and downstream acceptance. Attention weights, proximity and a generated CID are not proofs.

904 **I A semantic preservation contract for every translation**

905 The formal layer is not a bag of interchangeable solver adapters. For each source-to-target conversion
906 τ , record source domain, target logic/theory, vocabulary, variable/clock interpretation, assumptions,
907 unsupported constructs, omissions, source map and claimed preservation direction. Source truth may
908 be inferred from target truth only when that implication is established for the requested property. [N4,
909 N8]

$$\text{TargetChecker}(\tau(\varphi)) = \text{accepted} \not\Rightarrow \text{SourceSemantics} \models \varphi. \quad (3)$$

910 Table 15: Logic views, their useful roles, and their explicit scope.

911 Source view / 912 mechanism	Useful role	Scope to bind explicitly
913 Typed software IR / 914 first-order formulas	Contracts, alias/effect relations, claim dependencies.	Supported syntax and extractor semantics; unknown behavior stays possible.
915 F-logic / object-frame 916 views	Structured entities and relationships.	Actual engine/profile and translation tests; an advertised family is not an installed backend.
917 TDFOL / 918 temporal-deontic 919 view	Obligations, permissions, prohibitions and temporal constraints.	Clock/trace semantics, modal interpretation and property-preserving lowering.
920 DCEC / 921 cognitive-event or 922 event-calculus view	Actions, events, agents and knowledge/intention relations.	Domain axioms and supported modal/event fragment; no unqualified flattening to ordinary predicates.
923 SMT arithmetic / 924 arrays / bit-vectors	Finite checks, satisfiability, inductive or bounded obligations.	Selected theory, bit width, bounds, result type, solver and replay identity.
925 Lean / Coq / ATP + 926 reconstruction	Candidate proof search and independently checked theorems.	Proof payload, theorem/axioms, native checker version and exact reconstruction result.
927 ZK helper named 928 TDFOL_v1	Finite atom/implication derivation witness for an optional backend.	It accepts P and P→Q; this name does not imply full temporal, deontic or first-order semantics.

935 **I.1 Concrete loss test to add**

936 The inspected legacy converter maps $O(p)$ to p and a temporal operator applied to p to p ; it also
937 approximates a binary temporal formula by conjunction. Construct distinct source states or traces
938 in which the original modality differs while the projection agrees. The acceptance path must reject
939 lifting a target proof to the source claim unless an appropriate preservation condition is supplied. This
940 is a proposed semantic regression, not a newly executed repository test. [N8]

941 **I.2 Nonvacuity and contradictory premises**

942 Before accepting $G \Rightarrow A$ in component composition, check the intended consistency and reachability
943 conditions. Unsatisfiable G makes the implication vacuously true; it may expose a broken specification
944 rather than a correct component. Unknown is not inconsistent and not proved. Mutating axioms or
945 normative policy to close a proof is a new authorized specification change, not automatic repair.

946 **I.3 What can be proved about acceptance and reuse**

947 **I.3.1 Accepted-lineage invariant — proof sketch, not machine checked**

948 Fix a statement profile Θ and a policy predicate $P\Theta$ over artifact manifests. Assume: the initial
949 accepted root satisfies $P\Theta$; all required subject bytes and dependency references are verified; the
950 manifest is complete under an admitted completeness rule; each accepted evidence item satisfies its
951 designated checker relation; the sole publication operation enforces the expected parent and all gates;
952 and protected policy/checker identities cannot change under worker authority.

953 Then every root reachable solely through accepted publication transitions satisfies $P\Theta$. Proof:
954 induction on the number of successful publications. The initial case is assumed. For the induction
955 step, the publisher checks $P\Theta$ on the exact candidate manifest and current parent before its CAS
956 linearization; failed or stale attempts are not successors. The induction does not establish any property
957 omitted by $P\Theta$, truth of unsupported translations, honest signed observations, or atomicity of external
958 side effects.

959 **I.3.2 Dependency-projected test reuse — proof obligation**

960 Let $\text{Obs}\theta(t,s)$ be the relevant setup/call/teardown observation of test t in state s under a declared
961 runtime/effect profile θ . Suppose a complete projection πD and deterministic function $f\theta$ exist
962 such that $\text{Obs}\theta(t,s) = f\theta(\pi D(s))$ on all states admitted by the reuse policy. If $\pi D(s) = \pi D(s')$, equality
963 of the observations follows immediately. An admitted previous observation can therefore transfer
964 only within that profile and those assumptions. The difficult part is establishing completeness
965 and determinism of πD . Definitions alone miss fixture values; function coverage misses plugin,
966 clock, network, DB and teardown effects. Two equal digests of incomplete manifests do not satisfy
967 the premise. For effectful tests the profile must also show that required effects are replayed or
968 observationally unnecessary. Otherwise run setup and/or the test and teardown. A cold oracle and
969 adversarial mutations check the implementation but do not prove universal closure.

970 **I.3.3 Three acceptance questions, three different answers**

971 Behavioral claim: does candidate program/model satisfy its obligation?

972 Action permission: may this actor perform this action now?

973 Publication: does this candidate and its evidence become current?

974 Neither a proof of behavior nor a capability alone answers all three. Formalization determines
975 the statement; authorization binds actor, tool, arguments, resources, effects, scope, time and roots;
976 execution observations report what happened; publication chooses the accepted lineage. The reference
977 pre-dispatch code uses an in-memory consumption ledger and fake-dispatch defaults, so durable
978 real-tool integration requires its own adapter and restart evidence. [N7, N11]

979 **I.3.4 Useful autonomy requires positive-path witnesses**

980 Require a paired witness for each gate: a correctly authorized, well-bound case that progresses and a
981 minimally altered case that is rejected. Include a deny-all control in safety/cost diagnostics, not as a
982 competitive repair method. Otherwise default deferral, unavailable backends or adapter mismatches
983 can masquerade as perfect safety and low inference cost.

984 J Native proof code, metadata, and runtime observations

985 J.1 The inspected optional Groth16 path

986 The high-level adapter exposes Groth16/BN254 through a lazily imported Rust FFI backend and
987 checks an explicit enable flag. Generation defaults to circuit version 1; the version-2 TDFOL_v1
988 branch constructs a deterministic derivation trace and refuses a theorem outside its derivable fragment.
989 The helper's grammar is limited to named atoms, facts, and unary-antecedent implications. A Python
990 helper producing a trace is not by itself evidence that the deployed RICS enforces every intended
991 constraint. [N9–N10] The wrapper also supplies setup/ensure-setup functionality and allows a
992 seed. Reproducible research setup and test-vector determinism must remain separate from approved
993 operational key provenance. Existence of key files or a compiled binary is not a proof of key ceremony,
994 circuit soundness, correct deployment, or acceptable proving cost.

995 J.2 Bind inner statements to outer claims

996 The inspected verify_proof path forwards JSON decoded from proof_data to the FFI verifier. Metadata
997 and attestation views are also assembled around that object. At the consumer boundary, compare the
998 actual verified public inputs with the source, obligation, profile, environment, circuit, key and test-
999 inventory claims used for admission. Do not infer that every outer metadata field is cryptographically
1000 bound merely because the inner proof verifies. This is a required boundary test; no exploitable
1001 end-to-end attack was reproduced. [N9]

1002 Table 16: Adversarial changes at the native-proof admission boundary.

Adversarial modification	Expected disposition
1004 Change outer source/test root but keep proof_data unchanged.	Reject application claim if it differs from verified public inputs.
1006 Use proof of committed axioms as proof a Python test executed.	Reject mismatched statement/evidence class.
1008 Change circuit version, verification key, profile or revocation root.	Re-evaluate applicability; never silently reuse old admission.
1010 Omit one mandatory test from an otherwise valid batch.	Reject completeness even if every included leaf verifies.
1012 Pass call phase but fail teardown; reuse call with stale fixture values.	No whole-item pass; execute/reject as policy requires.
1014 Use simulated artifacts, auto-generated test keys or unknown backend.	No production credit; explicit unavailable/experimental outcome.

1016 J.3 Experiment still required

1017 Measure native generation and verification on a named circuit and key policy, not an estimated
1018 proof-shaped record. For test attestation, run a genuine runner with independent observations; for
1019 direct execution claims, name the encoded runtime and external-I/O assumptions. Report prover
1020 unavailable as unavailable. Recursive verification is optional; a manifest of individually checked
1021 proofs is a different and acceptable mechanism when labeled correctly.

1022 K Fill from one frozen, independently scored experiment

1023 Table 17: Main comparison; isolate component effects through matched
1024 ablations, not the combined arm alone.

Arm	Accepted solved / total	False accept / deny	Tokens / cost	Elapsed / human time
1027 A · raw + full validation	[TBD]	[TBD]	[TBD]	[TBD]
1028 B · semantic + same validation	[TBD]	[TBD]	[TBD]	[TBD]
1030 C · + symbolic route / reuse	[TBD]	[TBD]	[TBD]	[TBD]
1032 D · + governed lifecycle	[TBD]	[TBD]	[TBD]	[TBD]

1033 Table 18: Both sides of every acceptance boundary must be exercised.

Integration witness	Safe rejection	Valid progress	Evidence ID
1035 Pre-implementation provider gate	[TBD]	[TBD]	[TBD]

1036	Loss-aware formal translation	[TBD]	[TBD]	[TBD]
1037	Current native theorem/circuit profile	[TBD]	[TBD]	[TBD]
1038	Fixture/call/teardown reuse	[TBD]	[TBD]	[TBD]
1039	Root publication after restart	[TBD]	[TBD]	[TBD]
1040	Required world/procedure	[TBD]	[TBD]	[TBD]
1041	consumption			

1042 **K.1 Disclosure and reproducibility**

1043 Language models were used for [actual implementation assistance], [candidate goals/plans/patches],
 1044 [proof candidates], [data processing], and [manuscript preparation]. Record models, versions/dates,
 1045 settings, prompts or permitted summaries, data exposure and independent checks. Remove unused
 1046 roles rather than imply they occurred. Do not expose secrets, proprietary source, private witnesses or
 1047 private reasoning logs to satisfy a reproducibility checklist.

1048 **K.2 Final author actions**

1049 Freeze all landing worktrees and loaded dependency pins; reconcile implementations with ASEH,
 1050 DOEP, PCPR, PCSM and W1–W4; qualify the positive-path gate; bound each logic translation and
 1051 proof statement; regenerate result tables from raw evidence; report negative/missing results; replace
 1052 artifact labels with anonymous paths; remove author-only review notes and identifying links; use
 1053 the official workshop LaTeX style and check the actual PDF length. The separate audit/ledger is
 1054 author-only. No repository mutation, supervisor launch, native build or live experiment is implied by
 1055 this document revision.

1056 **L How symbolic computations change the next agent action**

1057 **L.1 Abstract state supplies candidate bounds**

1058 The inspected Python abstract interpreter uses a product of flat constants, integer intervals, nullness,
 1059 exceptions and effects. Its immutable lattice protocol exposes order, join, meet, widening and
 1060 narrowing; the adapter records local interprocedural summaries and source positions. The worklist
 1061 accumulates possible states at program points. Unsupported reflection, native calls, loading and
 1062 callbacks widen to opacity. A consumer may prune an impossible candidate only under a sound
 1063 admitted approximation, never from a learned low score. [N14]

$$\text{Abstract state} = \text{Const} \times \text{Interval} \times \text{Nullness} \times \text{Exceptions} \times \text{Effects}. \quad (4)$$

1064 **L.2 Interface composition creates a precise discharge job**

1065 A CompositionEdge references exact producer/consumer contracts and named normal, exceptional
 1066 and induction clauses. Unknown clause references are rejected before solver execution. The compo-
 1067 sition graph and receipt bind the semantic root, contracts, edges, obligation outcomes, uncovered
 1068 assumptions and SCCs. Its basic implication query is $G \wedge \neg A$. SAT provides a countermodel;
 1069 UNSAT provides solver-backed discharge of that encoded implication. Opaque prose stays uncovered.
 1070 Cycles need a typed invariant plus an independently solved closure check rather than mutually citing
 1071 guarantees. [N12]

$$G \wedge \neg A \text{ is UNSAT} \Rightarrow G \Rightarrow A \quad (\text{in the encoded theory}). \quad (5)$$

1072 This permits a caller to consume a dependency guarantee instead of repeatedly loading its body only
 1073 when current bindings, statement coverage and the consumer assumptions hold. A contract

1074 gap becomes a named repair or proof task. Consistency and reachability checks prevent treating an
 1075 impossible guarantee as meaningful implementation correctness.

1076 **L.3 CEGAR localizes abstraction error before code repair**

1077 The QF_LIA adapter searches a supplied Boolean abstraction of a control-flow graph and constructs
 1078 a named concrete path formula for each abstract error trace. A feasible path is retained as a

1079 real counterexample. A spurious path may be refined with a validated interpolant/core, weakest
 1080 precondition, or reviewed predicate. Iteration, predicate, abstract-state and trace-length budgets
 1081 are explicit; unavailable, unknown, timeout and budget exhaustion are separate dispositions. It is a
 1082 refinement adapter over existing contracts, not a second unrestricted model checker. [N13]

$$A \Rightarrow I, \quad I \wedge B \text{ is UNSAT}, \quad \text{Symbols}(I) \subseteq \text{Symbols}(A) \cap \text{Symbols}(B). \quad (6)$$

1083 Those are the checks required for an interpolant at the shared boundary. An unsat core identifies
 1084 relevant asserted clauses but is not automatically an interpolant or a kernel proof. After localization,
 1085 the supervisor may expand only the implicated context, request a bounded synthesis candidate, or
 1086 abstain. CEGIS changes candidate programs; CEGAR changes the abstraction. [D2, D5; N13]

1087 **L.4 Evidence feeds control, not just a report**

1088 Each result must be consumed by the actual plan/frontier or admission service: closed implication $\rightarrow$
 1089 reusable component contract; feasible counterexample $\rightarrow$ repair obligation; missing semantic input
 1090 $\rightarrow$ exact expansion; unavailable solver $\rightarrow$ deferral/fallback; complete current evidence $\rightarrow$ candidate
 1091 acceptance. A sidecar analysis that never affects these decisions cannot be credited with reducing
 1092 LLM use. Record the consumed result CID and subsequent control transition in the experiment.

1093 **M Composed control-loop specification**

1094 The following algorithm is retained from the preceding evaluated draft as an intended composition,
 1095 not a newly executed implementation result. It is a control-flow specification retained for comparison
 1096 with the final executable artifact.

1097 Algorithm 1: Intended composed supervision loop. Input: authorized q, current source forest F, policy
 1098 P, budget B

- 1099 1. reconcile event, task and artifact state; refresh F and derive G, O
- 1100 2. if cancelled or budget exhausted: return explicit non-success
- 1101 3. if every current obligation is admitted: return `verify_goal_completion()`
- 1102 4. generate bounded task proposals; preview and separately admit ready work
- 1103 5. if no task is admissible: return `wait_or_review()`, not goal success
- 1104 6. claim task; bind lease/fence, preimages, required context and validation
- 1105 7. choose deterministic, authorized residual-model, review or unavailable route
- 1106 8. if review/unavailable: record blocked attempt; return to the event service
- 1107 9. produce and apply candidate only through checked isolated execution
- 1108 10. rescan; derive `O_new` from authorized rules, not the worker's pass claim
- 1109 11. verify/reuse evidence; retain failed, unknown and pending obligations
- 1110 12. if refinable failure: record new residual; return bounded retry to service
- 1111 13. if evidence incomplete or unsafe: return `defer_or_reject()`
- 1112 14. persist manifest; recheck current bindings; CAS the expected parent
- 1113 15. if publication conflicts: return `reconcile()`, not completion
- 1114 16. record accepted task transition and updated F, G, O; reevaluate at line 1
- 1115 17. propose procedure/policy learning separately; never self-promote

1116 This algorithm specifies the composition to evaluate. Passing tests of isolated stop paths does not
 1117 establish the complete loop; qualification must exercise useful authorized progress as well as rejection.

1118 Paper checklist: completion pending The supplied workshop template includes a separate `checklist.tex`.
 1119 That questionnaire was not included in the uploaded template sources. This page is an explicitly

1120 marked compilation placeholder, not a reproduction of the organizers' checklist and not a declaration
1121 of compliance. [TODO] Obtain the official questionnaire, replace this file, and complete its answers
1122 against the final manuscript and frozen artifact. [TODO] Final experiment results, source snapshots,
1123 anonymous artifact access, and author-reviewed LLM-use disclosures remain pending. No checklist
1124 answer is inferred from the draft.